

BONUS LECTURE

Types of Inheritance in OOP & Java

Class 9 | Computer Science | NeverStop | By Sourabh Gorai

1. Single

2. Multilevel

3. Hierarchical

4. Multiple

5. Hybrid

Yeh Lecture Mein Kya Hai?

- Inheritance ka Quick Recap
- OOP ke 5 Types of Inheritance — definition + real life examples
- Java Scorecard — kaun se types supported hain
- Diamond Problem — kyun Multiple Inheritance Java mein banned hai
- Interfaces — Java ka smart jugaad to fix the problem
- Golden Rule of Java — ek line jo exam mein zaroor kaam aayegi

Quick Recap — Inheritance Kya Hai?

Inheritance matlab — **Child Class, Parent Class ki properties aur behaviors automatically le leti hai.** Iska matlab hai ki hume same code baar baar nahi likhna padta — bas inherit karo aur extend karo!

Example: Animal parent class ke paas eat() aur sleep() hai. Dog child class Animal se inherit karti hai — toh Dog ko automatically eat() aur sleep() mil jaata hai. Upar se Dog apna khud ka bark() bhi add kar leti hai!

```
Animal (Parent Class)
eat() sleep() breathe()
|
Dog (Child Class)
eat() sleep() breathe() + bark()
```

The 5 Types of Inheritance

01 Single Inheritance

Definition

Ek parent class → Ek child class.

Sabse simple aur common type. Child class parent ki saari properties inherit karti hai aur apni nai features add karti hai.

Real Life Example

Real Life: SmartPhone inherits from Phone.

Phone: call(), sendSMS()

SmartPhone: call(), sendSMS() +
browseInternet(), installApps()

```
graph TD
    Phone["Phone (Parent)"] --> SmartPhone["SmartPhone (Child)"]
```

Key Point: Java mein: FULLY SUPPORTED

02 Multilevel Inheritance

Definition

Chain of inheritance — generations ki tarah.

ClassA → ClassB → ClassC. ClassC ko automatically ClassA aur ClassB dono ki properties milti hain.

Real Life Example

Real Life: Dada ji → Papa → Tum.

Tum automatically dono generations se features inherit karte ho — ek chain ke through!

```
graph TD
    Vehicle["Vehicle → start(), stop()"] --> Car["Car → start(), stop(), openTrunk()"]
    Car --> ElectricCar["ElectricCar → start(), stop(), openTrunk(), chargeBattery()"]
```

Key Point: Java mein: FULLY SUPPORTED

03 Hierarchical Inheritance

Definition

Ek parent class → Multiple child classes.

Tree ki tarah branches nikalti hain. Ek parent, bahut saare children — sab parent ke features share karte hain.

Real Life Example

Real Life: Ek school mein ek principal — Science Dept, Arts Dept, Sports Dept sab school ki basic rules inherit karte hain.

```
graph TD
    Animal["Animal (Parent)"]
```

```
      / | \  
    Dog Cat Bird Fish  
bark() meow() fly() swim()
```

Key Point: Java mein: FULLY SUPPORTED

04 Multiple Inheritance

Definition

Ek child class → Do ya zyada parent classes.

Child class multiple parents ke features ek saath inherit karti hai.

Real Life Example

Real Life: FlyingCar — Car se bhi inherit karta hai (drive, brake) aur Airplane se bhi (fly, land). Dono ke features ek saath!

```
Car (Parent A) Airplane (Parent B)
  drive(), brake() fly(), land()
      \ /
      \ /
    FlyingCar (Child)
  drive(), brake(), fly(), land()
```

DIAMOND PROBLEM — YEH HAI VILLAIN!

```
ClassA → show()
      / \
    ClassB ClassC
show() override show() override
      \ /
    ClassD
show() — KAUN SA USE KAREIN?!
```

Problem yeh hai: ClassD inherit karta hai ClassB aur ClassC dono se. ClassB ne show() apne tarike se override kiya. ClassC ne bhi show() alag tarike se override kiya. Ab jab ClassD show() call karta hai — **compiler confuse ho jaata hai ki kaunsa version use karein?** Yahi ambiguity hai jise Diamond Problem kehte hain!

Ghar Wali Analogy

Tum ghar aake puchte ho — "Aaj dinner mein kya hai?"

Mummy: "Dal Chawal!" Papa: "Roti Sabzi!"

Ab tum confuse — kisko maanein? Yahi hai Diamond Problem!

Key Point: Java mein Multiple Inheritance using classes: NOT SUPPORTED

05 Hybrid Inheritance

Hybrid = Do ya zyada types ka combination. Jaise Hierarchical + Multiple ek saath.

Kyunki Hybrid mein almost hamesha Multiple Inheritance involve hoti hai — **Diamond Problem yahan bhi aata hai.** Isliye Java mein Hybrid Inheritance bhi directly allowed nahi hai classes ke through.

```
ClassA
 /  \
ClassB ClassC
  \  /
ClassD
(Diamond shape = Diamond Problem!)
```

Key Point: Java mein Hybrid Inheritance using classes: NOT SUPPORTED

Java Ka Scorecard

#	Type	Java Support?	Reason
1	Single	✓ YES	Simple, no ambiguity
2	Multilevel	✓ YES	Chain, no ambiguity
3	Hierarchical	✓ YES	One parent, no ambiguity
4	Multiple	✗ NO	Diamond Problem
5	Hybrid	✗ NO	Involves Multiple Inheritance

Interfaces — Java Ka Jugaad!

Interface kya hota hai? Ek **contract** — jo sirf batata hai ki yeh yeh kaam KARNA PADEGA, but HOW karna hai nahi batata. Actual code likhna class ka kaam hai.

Real Life — Job Contract Analogy	Power Socket Analogy
Company ka contract kehta hai: • Office time pe aana PADEGA • Friday tak kaam submit KARNA PADEGA • Weekly meeting attend KARNI PADEGI Contract sirf rules batata hai — tune khud decide karna hai HOW to do it. Yahi Interface karta hai!	Wall ka socket ek interface hai — standard shape aur electricity provide karta hai. Phone charger, laptop, fan — sab plug kar sakte hain as long as they follow the standard. Har device electricity alag tarique se use karta hai — but sab same socket contract follow karte hain!

Interface Ka Code — Java Mein

Interface mein sirf method **declaration** hoti hai — koi body nahi. **implements** keyword se ek class multiple interfaces use kar sakti hai!

```
// Interface — sirf declaration, koi body nahi
interface Flyable {
    void fly();
    void land();
}

interface Drivable {
    void drive();
    void brake();
}

// Class MULTIPLE interfaces implement kar sakti hai!
class FlyingCar implements Flyable, Drivable {
    public void fly() { System.out.println("FlyingCar is flying!"); }
    public void land() { System.out.println("FlyingCar is landing!"); }
    public void drive() { System.out.println("FlyingCar is driving!"); }
    public void brake() { System.out.println("FlyingCar braking!"); }
}
```

Diamond Problem Solve Kyun Ho Jaata Hai?

Diamond Problem tha kyunki dono parent classes ke paas apni apni method body thi — compiler confuse tha ki kaunsi use karein.

Interface mein koi method body hoti hi nahi! Sirf declaration hoti hai. Toh jab class dono interfaces implement karti hai — class khud ek hi version likhti hai. Koi confusion nahi!

```
interface InterfaceA { void show(); } // Koi body nahi
interface InterfaceB { void show(); } // Koi body nahi

// Bilkul valid Java code!
class MyClass implements InterfaceA, InterfaceB {
    public void show() {
        // Sirf EK version — MyClass ne khud likha
        System.out.println("MyClass ka show()");
    }
}
```

Wahi Dinner Analogy — Ab Solved!

Mummy bhi kehti hai: **"Dinner ka kuch plan karo."** Papa bhi kehte hain: **"Dinner ka kuch plan karo."**

Dono ne sirf requirement bataya — koi specific jawab nahi diya. Ab TUM khud decide karo — koi conflict nahi!

Class vs Interface — Side by Side

Feature	Class (extends)	Interface (implements)
Method Body?	YES — hoti hai	NO — sirf declaration
Multiple Parents?	NO — ek hi	YES — kitni bhi
Keyword	extends	implements
Variables?	YES — hote hain	Only constants (final)
Diamond Problem?	YES — hota hai	NO — nahi hota
Used For?	Code & behavior share	Contract / Blueprint

GOLDEN RULE OF JAVA

"Java mein tum sirf **EK class extend** kar sakte ho —

but JITNI chahein utni interfaces implement kar sakte ho!"

`extends` → sirf ek | `implements` → multiple allowed